

Teknisk beskrivning JobChaser

JobChaser är en högpresterande fullstack-applikation byggd med fokus på typsäkerhet, skalbarhet och säker datahantering. Systemet är arkitektoniskt uppdelat i en klientapplikation (SPA) och ett RESTful API.

Arkitekturöversikt

Systemet följer en klassisk **Client-Server-arkitektur** där frontend och backend kommunicerar via JSON-baserade API-anrop. Genom att använda TypeScript genomgående i hela stacken uppnås "*End-to-End Type Safety*", vilket innebär att datastrukturer i databasen speglas hela vägen upp till UI-komponenterna.

Frontend & State Management

- **React 19 & Vite:** Utnyttjar den nya *use-hooken* och förbättrad rendering för att minimera laddtider.
- **Zustand (Global State):** Implementerar en lättviktig *store* för att hantera globala tillstånd som *authUser*. Genom att använda en *persist-middleware* säkerställs att användarsessionen kvarstår vid sidomladdningen genom att synkronisera applikationens state med *localStorage* via en s.k. *hydration-process*.
- **useReducer & Context API (Filtrering):** Hanterar den mer komplexa filtreringslogiken (sökterm, ort, kategori) genom ett förutsägbart reducer-mönster. Detta möjliggör en centraliserad hantering av filter-actions som delas globalt.
- **Tailwind CSS 4:** Drar nytta av den nya arkitekturen i version 4 för snabbare builds och en renare CSS-variabelhantering.

Backend & Databaslager

- **Drizzle ORM:** Valdes för dess minimala runtime-overhead och förmåga att skriva SQL-liknande queries med full typsäkerhet mot PostgreSQL.
- **Zod Integration:** Används som ett enhetligt valideringslager. Samma scheman används för:
 1. Inkommande JSON i Request Body (Runtime-check).
 2. Inferens av TypeScript-typer för frontend-formulär.
 3. Validering i Drizzle-modeller.

Säkerhet & Autentisering

- **JWT-baserad auktorisering:** Vid inloggning utfärdas en signerad *token* som lagras i klienten. Denna skickas med i *Authorization-headern* för att ge åtkomst till skyddade resurser.
- **Password Hashing:** Använder *bcrypt* med en *salt round* på 10 för att skydda användardata mot brute-force-attacker vid eventuellt databasläckage.
- **Protected Routes:** Implementerade via React Router 7, där en "Higher Order Component" kontrollerar autentiseringsstatus innan känsliga sidor renderas.

Databasmodell (ER-diagram)

Systemet använder en relationell databasmodell för att bibehålla dataintegritet.

- **Users:** Lagrar autentiseringsuppgifter och användarprofiler.
 - **Jobs:** Innehåller all metadata om jobbannonser (titel, företag, beskrivning, etc.). En "One-to-Many" relation kopplar jobb till en skapare (User).
 - **Saved:** En kopplingstabell (Join Table) som hanterar "Many-to-Many"-relationen mellan användare och jobb, vilket möjliggör funktionaliteten för sparade annonser.
-

Implementering av features (extrauppgifter)

Utöver grundkraven har applikationen utökats med avancerad funktionalitet för att efterlikna ett modernt produktionssystem. Följande arkitektoniska och funktionella tillägg har implementerats:

Avancerad Autentisering & Säkerhet - Säkerheten är implementerad i flera lager för att minimera sårbarheter:

- **Datavalidering:** Zod används för strikt schema-validering av användaruppgifter (t.ex. lösenordskrav på minst 6 tecken) redan innan data når databaslagret.
- **Auktorisering (Skyddade rutter):** Frontend skyddas via en *ProtectedRoute*-komponent (Higher-Order Component) som förhindrar obehörig åtkomst till specifika vyer. Backend säkras genom en dedikerad *authenticateToken*-middleware som verifierar JWT-signaturen på skyddade endpoints.
- **Kryptografi:** Användarlösenord saltas och hashas med *Bcrypt* innan lagring (Data at Rest), vilket skyddar mot kompromettering vid ett eventuellt databasintrång.
- **Strategisk Global State-hantering (Zustand)** - För att undvika "prop-drilling" och skapa en skalbar frontend-arkitektur har det globala tillståndet delats upp enligt *Single Responsibility Principle (SRP)*:
- **AuthStore:** En isolerad store som hanterar sessionen (*user*, *token*, *isAuthenticated*). Genom Zustands persist-middleware synkroniseras denna state med *localStorage*, vilket möjliggör "hydration" och bibehåller inloggningen vid sidomladdning (page reload).

Databasrelationer & Favoritmarkeringar (Sparade Jobb) - Applikationen stödjer ett komplext dataflöde där användare kan spara både lokala annonser och jobb från externa API-källor.

- **Relationsmodell:** Implementerat via en klassisk *Many-to-Many*-relation i PostgreSQL. En kopplingstabell (*saved_jobs*) mappar *user_id* mot specifika jobb, vilket säkerställer dataintegritet.
- **UX:** En dedikerad vy (*/saved*) ger användaren full kontroll över sina sparade favoriter, vilket skapar en mer personlig och engagerande användarupplevelse.

Dynamiskt Tema (Dark Mode) - Ett fullt integrerat Dark Mode har skapats för att möta moderna tillgänglighets- och designkrav.

- **Persistens:** Användarens preferens lagras i *localStorage* för att säkerställa att rätt tema laddas direkt vid återbesök (förhindrar s.k. *Flash of Unstyled Content*).
- **Teknisk implementation:** Utnyttjar Tailwind CSS inbyggda *dark*: modifierare för att effektivt togglar färgpaletter över hela gränssnittet från en global kontext.

Fullständig CRUD-arkitektur via REST Systemet erbjuder kompletta CRUD-operationer (Create, Read, Update, Delete) enligt REST-principer.

- **Rättighetskontroll:** Affärslogiken säkerställer att en användare endast kan uppdatera (via *PATCH* för partiella uppdateringar) eller radera *sina egna* jobbannonser.

Reflektion, styrkor och brister

Detta projekt har verkligen hjälpt mig att få förståelse för hela stacken och hur frontend kommunicerar med backend och de diverse bitar som krävs för att båda ska kunna göra sitt jobb. Detta är det bredaste projektet jag gjort hittills där jag gjort allt från grunden, från databasen med DRIZZLE till auktoriseringen med JWT tokens. Jag fick testa Zustand för förstå hur det kan vara bättre alternativ för *useContext*. Detta har fått mig att förstå att det inte en applikation inte är så enkelt gjord som det ibland kan se ut på ytan, utan bakom ytan gömmer det sig en hel del logik och hantering av data.

Viktiga insikter:

- **Arkitektur:** Jag har lärt mig hur avgörande kommunikationen mellan frontend och backend är för systemets stabilitet. Att implementera **Zod** genom hela stacken har visat mig fördelarna med strikt typsäkerhet för att eliminera buggar tidigt i flödet.
- **State Management:** Att använda **Zustand** istället för *useContext* gav mig en djupare förståelse för hur man hanterar globalt tillstånd på ett skalbart och prestandaoptimerat sätt.
- **Säkerhet & Data:** Projektet har lärt mig vikten av dataintegritet och hur man hanterar känslig information genom kryptering och säkra middlewares.

Sammanfattningsvis har projektet förvandlat min syn på webbutveckling: från att bygga isolerade komponenter till att konstruera sammanhängande, robusta system där varje tekniskt beslut påverkar både säkerhet och användarupplevelse.

Styrkor

- **End-to-End Typsäkerhet:** Kombinationen av TypeScript, Zod och Drizzle ORM eliminerar diskrepanser mellan klient och server. Genom att ha en "Single Source of Truth" för datastrukturer minimeras körtidsfel avsevärt.
- **Prestandaoptimering:** React 19 och Zustand samverkar för att skapa ett reaktivt gränssnitt med minimal overhead. På backendsidan erbjuder Drizzle ORM rå SQL-prestanda kombinerat med ORM-bekvämlighet vilket är betydligt snabbare än tyngre alternativ.

- **Användarcentrerad Design:** Implementationen av persistent dark mode och hanteringen av sparade jobb höjer den upplevda kvaliteten och gör applikationen mer "levande" och anpassningsbar.
- **Kombinerar lokalt och hämtat API:** Har lyckats sammanväva ett yttre API från arbetsförmedlingen med min egna lokala databas så att de lever ihop sömlöst.
- **Separation of Concerns:** Den tydliga separationen mellan routing (controllers), databasinteraktion (Drizzle-schema) och UI-presentation gör kodbasen modulär, lättläst och framtidssäkrad för skalning.

Brister & Framtida Förbättringspotential

- **Säkerhetsrisk med Token-lagring:** För närvarande lagras JWT i *localStorage*. Även om detta är vanligt i skolprojekt, exponerar det applikationen för *Cross-Site Scripting (XSS)*-attacker. I en produktionsmiljö bör tokens flyttas till **HttpOnly & Secure Cookies** för att helt skydda mot attacker.
- **Sessionshantering (Avsaknad av Refresh Tokens):** Access-tokens går ut efter en timme. I dagsläget tvingas användaren logga in på nytt, vilket skadar UX. En robust lösning hade varit att implementera "Silent Authentication" via en roterande *Refresh Token*.
- **Informationsläckage i Felhantering:** Backend-loggar och vissa felmeddelanden (särskilt i auth-middlewären) returnerar för närvarande för mycket teknisk kontext till klienten. Detta bryter mot principen om minsta möjliga information (*Information Disclosure*) och bör abstraheras bort i produktion.
- **Server Side Rendering:** Migrering av vissa nuvarande rutter till en hybrdlösning för bättre SEO på publika jobbannonser.
- **WebSockets:** Implementera realtidsnotiser när en användare får svar på en annons.
- **Filuppladdning:** Göra det möjligt för användare att spara sitt CV och Personliga brev på sin sitt privata konto. För att underlätta processen under ansökan.